

9: Differentialligninger af 1. orden. Runge-Kuttas metode

Vi har i afsnit 8 set hvorledes en differentialligning af typen

$$\frac{dy}{dx} = h(x, y) \quad (9.1)$$

løses ved hjælp af Eulers metode. I appendiks D er beskrevet nogle forbedrede metoder, der tillader, at man kan tage flere skridt med større steplængde Δx , før der bliver fare for afsporing. En rigtig god og meget anvendt standardmetode er den såkaldte Runge-Kuttas metode af 4. orden. Som vist i fig. 9.1 er det i FPRO særdelses let at bringe denne metode i anvendelse.

Sædvanligvis kan man gå ud fra, at en differentialligning af typen (9.1) har én, og kun én, løsning gennem et givet punkt (x_0, y_0) . Dette synes ganske oplagt, når man tænker på den numeriske integrationsmåde, således som den er forklaret i starten af afsnit 8 - naturligvis forudsat at $h(x, y)$ er defineret i (x_0, y_0) . Matematikken byder imidlertid på spidsfindigheder, der kan drille i endog meget simple sammenhænge. Fig. 9.2 byder på et eksempel. Den teoretiske baggrund for dette er følgende.

Til tiden $t = 0$ slippes en sten ud for $s = 0$ på en lodret nedadgående s -akse. Idet tyngdeaccelerationen betegnes g , vil positionen til tiden t være givet ved

$$s = \frac{1}{2}gt^2 \quad (9.2)$$

Man kan betragte denne funktion som den løsning til differentialligningen¹

$$\frac{ds}{dt} = \sqrt{2gs}, \quad (9.3)$$

der opfylder begyndelsesbetingelsen $s(t) = 0$ for $t = 0$. I fig 9.2 er vist, hvad der sker, når man løser differentialligningen numerisk: I stedet for løsningen (9.2) får man s konstant lig nul!! Dette strider jo ganske mod vor fysiske forventning, men ved nærmere eftertanke må vi erkende, at den numerisk bestemte løsning er matematisk uangribelig: *To forskellige løsninger opfylder den givne begyndelsesbetingelse, og vore numeriske metoder finder den mest simple.* Man kan ikke altid ignorere differentiallignings-teoriens eksistens- og entydighedssætninger².

I figurene 9.3 og 9.4 beskrives nogle små praktiske forbedringer af programmet i fig 9.1.

¹ Differentialligningen kan f. eks. opnås ved at sætte vundet kinetisk energi lig med tabt potentiel energi,

$$\frac{1}{2}mv^2 = mgs; v = \frac{ds}{dt} \quad (a)$$

forkorte massen væk, gange 2 over og tage kvadratrods. Vi bemærker, at vi ved at differentiere en ekstra gang på begge sider af (9.3), og bortforkorte m samt løsningen $v = 0$, får

$$\frac{dv}{dt} = g \Leftrightarrow \frac{d^2s}{dt^2} = g \quad (b)$$

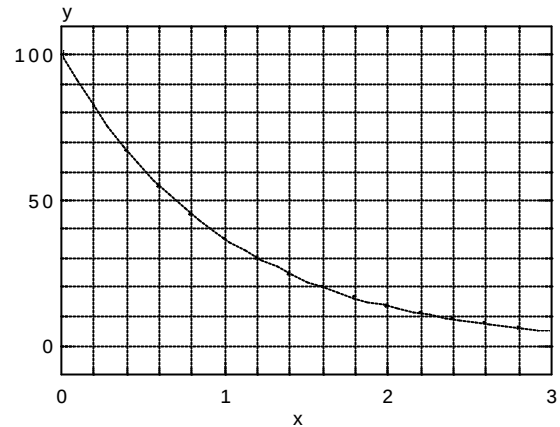
hvilket er den sædvanlige 2.ordens differentialligning for det frie fald.

Den her beskrevne behandling af det frie fald kan virke noget søgt. Dens princip er imidlertid standard overfor visse problemstillinger. Tænk f. eks. på en bungee-jumper, der springer ud i en tung elastik: Hvorledes påvirker elastikken det frie fald? På højre side af (a) må vi addere dén potentielle energi, som elastikken har tabt, når jumbere er faldet stykket s . Når differentialligningen (9.3) er ændret tilsvarende, beskriver den faldet - med to løsninger gennem $(0,0)$. En anden-ordens differentialligning uden denne tvetydighed kan opnås ved en ekstra differentiation som i (b). Beregningerne kan findes i David Kagan og Alan Kott's artikel: *The Greater-Than-g Acceleration of a Bungee Jumper* (*The Physics Teacher* Vol. 34 Sept. 1996 s. 368).

² Man kan finde en formulering af disse for 1. ordens ligninger i Erwin Kreyszig: *Advanced Engineering Mathematics* (John Wiley, 7. ed. 1993, side 54).

Værdi

	Navn	Udtryk	Værdi	Tekst
2	xmin		0	Koordinat-
3	xmax		3	systemets
4	ymin		-10	ud-
5	ymax		110	strækning
6				
7	x0		0	Start-x
8	xslut		100	Slut-x
9	y0		3	Start-y
10				
11	Dx		0.2	
12	n		20	Step pr. Dx
13	dx	Dx/n	2	Steglængde
14				
15	k		1	konstanten i lign

**Klar**

```

Model Diff ligning
  y':=-k*y
endmodel
IntegrateMethod := RK4 {RK2, Euler}

```

```

x:=x0
y:=y0
UpdateRate := n

```

Løkke

```

Integrate Diff ligning(x,dx)
x:=x+dx
If x>xslut then stop

```

Søjler 1

	1	2	3
			yeksakt
	x	y	100*exp(-k*x)
0	0.0	100.0000	100.0000
1	0.2	81.8733	81.8731
2	0.4	67.0324	67.0320
3	0.6	54.8817	54.8812
4	0.8	44.9335	44.9329
5	1.0	36.7885	36.7879
6	1.2	30.1200	30.1194
7	1.4	24.6602	24.6597
8	1.6	20.1902	20.1897
9	1.8	16.5304	16.5299
10	2.0	13.5340	13.5335
11	2.2	11.0807	11.0803
12	2.4	9.0721	9.0718
13	2.6	7.4277	7.4274
14	2.8	6.0813	6.0810
15	3.0	4.9789	4.9787

Figur 9.1: Program *MAT09a.FPR* finder her, ved *Runge-Kuttas metode af 4. orden*, den løsning til differentialligningen

$$\frac{dy}{dx} = -ky; k = 1$$

som starter i $(x_0, y_0) = (0, 100)$. De fundne punkter (x_i, y_i) plottes sammen som små firkanter sammen med den fuldt optrukne graf for den eksakte løsningsfunktion $f(x) = 100 \exp(-x)$. På grafen kan resultatet ikke skelnes fra det, vi i fig. 8.3 fandt ved viser imidlertid en langt bedre overensstemmelse end den tilsvarende tabel i hjælp af Eulers metode med en steplængde for integration på $0.2/20 = 0.01$. Tabellen fig. 8.3 - til trods for at steplængden dx nu er 20 gange så lang (nemlig lig steplængden $Dx = 0.2$ for udskrift til graf/tabel fremfor lig $Dx/20 = 0.01$ som i fig. 8.3).

Bemærk at man kan vælge frit mellem 3 forskellige integrationsmetoder. Udskiftes *RK4* i *IntegrateMethod :=*, så integrerer programmet med Runge-Kuttas metode af 2. orden. Udskiftes med Euler, så integrerer programmet med Eulers metode (som er af 1. orden).

Værdi

	Navn	Udtryk	Værdi	Tekst
2	tmin		0	Koordinat-
3	tmax		3	systemets
4	smin		-2	ud-
5	smax		40	strækning
6				
7	t0	Dt	2	Start-tid
8	tslut		100	Slut-tid
9	s0	$g \cdot Dt^2/2$	0.2	Start-s
10				
11	Dt		0.2	
12	n		1	Step pr. Dt
13	dt	Dt/n	0.2	Steplængde
14				
15	g		9.82	m/s ²

Klar

Model Difffligning

 $s' := \sqrt{2 \cdot g \cdot s}$

endmodel

IntegrateMethod:= RK4 {Euler, RK2}

t:=t0

s:=s0

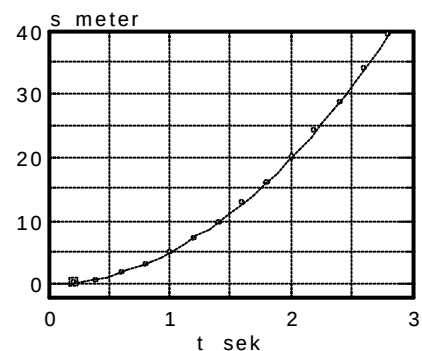
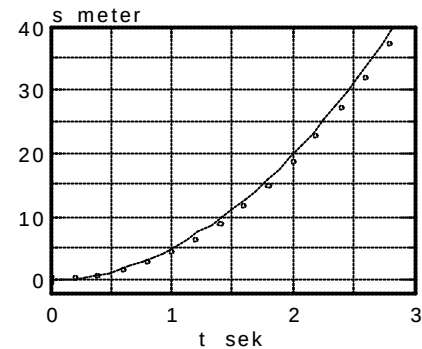
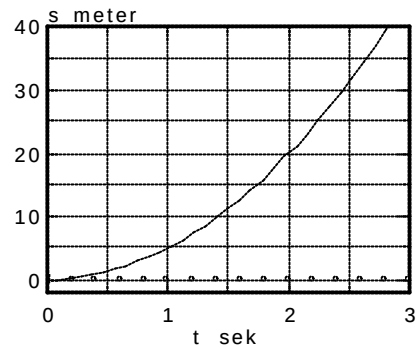
UpdateRate:=n

Løkke

Integrate Difffligning(t,dt)

t:=t+dt

If t>tslut then stop



Figur 9.2: Program *MAT09b.FPR*. De tre grafer er resultaterne af forskellige forsøg på at finde den løsning til differentialligningen for det frie fald

$$\frac{ds}{dt} = \sqrt{2gs} \quad ; g = 9.82 \frac{m}{s^2},$$

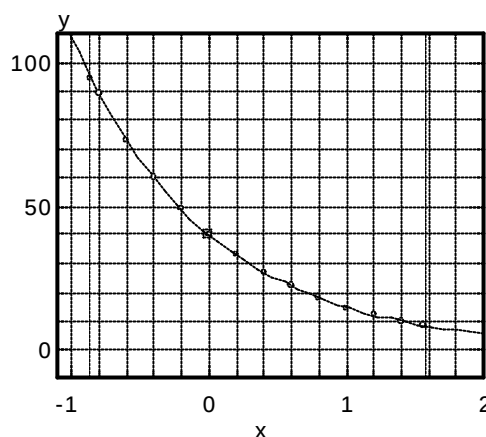
som starter i $s_0 = 0$ til tiden $t_0 = 0$. Integrationen sker ved Runge-Kuttas metode af 4. orden med steplængde på $dt = Dt = 0.2$ sek, og de fundne punkter (t_i, s_i) er plottet som små cirkler sammen med den fuldt optrukne graf for den eksakte løsningsfunktion $s(t) = \frac{1}{2}gt^2$. Som forklaret i teksten er der faktisk endnu en eksakt løsning gennem $(0,0)$, nemlig $s(t) = 0$, og i øverste graf, er det åbenbart den, programmet har fundet. I næste graf er begyndelsesbetingelsen ændret en anelse, således at starten sker i $s_0 = 1$ mm til tiden $t_0 = 0$. Fysisk skulle denne lille ændring ikke kunne ændre bevægelsen nævneværdigt, men matematisk er effekten markant: Programmet springer næsten helt over til den anden af de to løsninger.

I sidste graf er (t_0, s_0) valgt til at være punktet $(dt, \frac{1}{2}g dt^2) = (0.2 \text{ sek}, 0.2 \text{ meter})$ på den eksakte løsningsgraf. Med denne start forløber integrationen næsten præcist oven i den eksakte løsningsgraf.

Læseren kan afprøve de samme muligheder med Eulers metode fremfor RK4. Resultatet viser samme klare tendens, blot med større afvigelser i 2. og 3. graf (med samme dt).

Værdier

	Navn	Udtryk	Værdi	Tekst
2	xmin		-1.1	Koordinat-
3	xmax		2	systemets
4	ymin		-10	ud-
5	ymax		110	strækning
6				
7	a		-0.879	Venstre interv
8	x0		0	Start- x
9	b		1.573	Højre interval
10	y0		40	Start - y
11				
12	Dx		0.2	
13	n		10	Step pr. Dx
14		Dx/n	0.02	Steplængde
15				
16	k		1	konstanten



Klar

```

SelectWindow(Graf 1)
ClearLines
Line(a,ymin,a,ymax)
Line(b,ymin,b,ymax)

```

Model Diff ligning

```

y':=-k*y
endmodel
IntegrateMethod:= RK4

```

```

x,y,dx:=x0,y0,Dx/n

```

```

s:=1

```

Løkke

```

If s=2 then s,x,y,dx:=1,x0,y0,-dx
i:= 0
If x > b-dx then s:=2
If x < a-dx then s:=3
If s=1 then
  Integrate Diff ligning(x,dx)
  x:=x+dx
Endif
i:=i+1
Until i = n or s > 1
If s = 3 then stop

```

Søjler

	1	2	3	4
Tekst				
Navn			yeksakt	
Enhed				
Udtryk	x	y	$y_0 \cdot \exp(-k \cdot x)$	s
0	0.00	40.0000	40.0000	1
1	0.20	32.7492	32.7492	1
2	0.40	26.8128	26.8128	1
3	0.60	21.9525	21.9525	1
4	0.80	17.9732	17.9732	1
5	1.00	14.7152	14.7152	1
6	1.20	12.0478	12.0478	1
7	1.40	9.8639	9.8639	1
8	1.57	8.3218	8.3218	2
9	-0.20	48.8561	48.8561	1
10	-0.40	59.6730	59.6730	1
11	-0.60	72.8848	72.8848	1
12	-0.80	89.0216	89.0216	1
13	-0.87	95.4764	95.4764	3
14				

Figur 9.3: Program *MAT09c.FPR*. Vi har i fig. 9.1 beskrevet, hvorledes differentialligningens løsning forløber til højre for det givne punkt (x_0, y_0) . Forløbet mod venstre kan fås ved at vælge dx negativ. I figur 9.3 er programmet fra fig. 9.1 modificeret, så det giver løsningens forløb henover et interval $[a, b]$, der strækker sig til begge sider af x_0 . En lille detalje: Da programmet selv skal skifte fortegn for dx , så er dx nu defineret som en løkke-variabel, og ikke som en enkeltværdi-variabel som i fig. 9.1.

Bmærk stop-ordren *Until i=n or s>1* i den Repeat-løkke, der nu har erstattet ordren *Updaterate := n*. Så længe "job-bestemmeren" s er lig 1, skal denne indre løkke gennemløbes n gange, inden programmet fortsætter til hovedløkkens sidste ordre, efter hvilken der udskrives til tabel/graf. Når Repeat-løkkens gentagne integrationer, når frem til et af endepunkterne i intervallet $[a, b]$, så skifter s værdi. Det betyder, at dersom udskrivningspunkterne $\dots, x_0 - Dx, x_0, x_0 + Dx, x_0 + 2Dx$, ikke rammer et intervalledepunkt præcist, så fås en ekstra udskrivning af det x i intervallet, der kommer tættest på i de små integrationsstep dx .

Søjler Søjle 3 er ikke vist. Den er som i fig. 9.3

Klar

Som i *MAT09c.FPR* (fig. 9.1)
bortset fra at dettes sidste linje er
udvidet fra $s:=1$ til $s,m:=1,0$

Løkke

If $s=2$ then $s,x,y,dx:=1,x_0,y_0,-dx$

$i:=0$

Repeat

* If $x > b-dx$ then $s,mb:=2,m+1$

If $x < a-dx$ then $s,ma:=3,m+1$

If $s=1$ then

Integrate Difflligning(x,dx)

$x:=x+dx$

Endif

$i:=i+1$

Until $i=n$ or $s>1$

$m:=m+1$

If $s=3$ then

$xn[0]:=x$

$yn[0]:=y$

For $i:=1$ to $ma-mb-1$ Do =====>

$xn[i]:=xx[ma-i]$

$yn[i]:=yy[ma-i]$

Next

For $i:=0$ to mb Do

$xn[ma-mb+i]:=xx[i]$

$yn[ma-mb+i]:=yy[i]$

Next

Stop

Endif

	1	2	4	5	6	7
Tekst						
Navn	xx	yy			X	Y
Enhed						
Udtryk	x	y	s	m	xn	yn
0	0.00	40.0000	1	0	-0.87	95.4764
1	0.20	32.7492	1	1	-0.80	89.0216
2	0.40	26.8128	1	2	-0.60	72.8848
3	0.60	21.9525	1	3	-0.40	59.6730
4	0.80	17.9732	1	4	-0.20	48.8561
5	1.00	14.7152	1	5	0.00	40.0000
6	1.20	12.0478	1	6	0.20	32.7492
7	1.40	9.8639	1	7	0.40	26.8128
8	1.57	8.3218	2	8	0.60	21.9525
9	-0.20	48.8561	1	9	0.80	17.9732
10	-0.40	59.6730	1	10	1.00	14.7152
11	-0.60	72.8848	1	11	1.20	12.0478
12	-0.80	89.0216	1	12	1.40	9.8639
13	-0.87	95.4764	3	13	1.57	8.3218
14						

Spørgsmål: Kunne man ikke undlade linjerne

$xn[0]:=x$

$yn[0]:=y$

ved at lade For - to - Do løkken starte med $i:=0$? .

Svar: Nej. Programmet stopper så med melding om indexfejl i $xx[ma-i]$.

Figur 9.4: Programmet *MAT09d.FPR* laver først præcist det samme som *MAT09c.FPR* i fig. 9.3 - tallene i søjlerne 1, 2, 3 og 4 er uændrede. Det afslutter imidlertid med at omgruppere de fundne punkter (x, y) således at de i tabellen - under (X, Y) - er ordnet efter stigende værdier af X . For at opnå dette, har vi indført tallet m , der starter med værdien $m = 0$ og vokser med 1 for hvert fulde gennemløb af *Løkke* - dvs. for hver gang der sker udskrift til tabellen. I denne bliver m således identisk med række nummeret yderst til venstre. Parametren mb (ma) noterer den værdi m har, når programmet udskriver værdien af y ved højre (venstre) intervalendepunkt. I *Søjler* ovenfor er disse værdier vist med fede typer ($mb = 8$ og $ma = 13$). En detalje: Bemærk at der i den med * mærkede *If* - sætning står $mb = m+1$ fremfor $mb = m$. Dette skyldes, at mb skal notere den værdi m har ved slutningen af *Løkke*. Og m opskrives jo med 1 umiddelbart efter Repeat-løkkens *Until*.